Measuring Whether Behavioural Rules Change Model Decisions

A 12-round empirical study on a two-model local system

Michal Malček, with Claude Opus 5 and a local Gemma-4 12B · August 2026

Abstract

Behavioural rules for LLM agents are usually written, stored, and assumed to work. We measured whether ours actually do. Across 12 rounds on a working two-model system we tested three questions that are normally conflated: can a rule be **retrieved** when it is needed, does it **change** what the model decides, and does the model **know** which of its rules help it.

The results were not what we expected on any of the three. Retrieval by rule wording succeeded 1 time in 5; the same rules stored as descriptions of the *situations* they govern reached 10/10. Of 16 rules, 10 changed nothing for either model — they were already the model's own standard — while 6 measurably changed a decision, and **none made a decision worse**. And when asked directly which of its rules carried real information, the model misclassified 7 of 8, including three that had demonstrably changed its behaviour minutes earlier.

The last finding is the methodological one: **a model can follow a rule without being able to judge it.** Rule value has to be measured behaviourally, through A/B comparison, never by asking.

1. Setting

The system is a personal AI environment ("the hive") shared by three participants: one human, one large cloud model (Claude Opus 5, referred to below as the *cloud agent*), and one local model (Gemma-4 12B Q5_K_M running on llama.cpp, referred to as the *local agent*) that performs long agentic runs, coding, and retrieval — not a chat assistant.

Shared state lives in Postgres (pgvector + Apache AGE) and Qdrant: ~1,100 memory records, ~1,000 knowledge-graph entities, ~3,000 code entities with ~10,000 relationships. Everything runs on one 24 GB Apple M4; no cloud inference for the local agent.

The practical problem: both agents need a shared set of operating rules, but the human's constraints ruled out the obvious designs. Rules must not be enforced at every step, must not become a dead list nobody reads, and must earn their place rather than accumulate.

2. What we tested and what failed

Seven candidate designs were rejected, each on evidence rather than preference. The rejections are more informative than the final design, so they come first.

2.1 Rules in the system prompt — rejected

Standard practice, and it fails for a structural reason. The starting state was measured before anything was designed, and it was worse than assumed:

where the rules lived	count
`rule-*.md` files on disk	13
`## Rule` sections inside one anchor document	10
records tagged as rules in the vector store	1

None of the three was authoritative. The anchor claimed that 11 standalone files had been consolidated into it; the files still existed. Duplication with no source of truth.

Two rules had never made it into the anchor at all — including, with some irony, *"suspect the measurement before the thing measured."* It was indexed, it won a search for its own name, and it was absent from the document that actually loads at session start. It existed and was not where decisions happen.

The structural defect: a 16 KB anchor containing 11 rules produces **one averaged embedding**. That vector represents the mean of sixteen different ideas, so it wins for none of them. Measured: 5 of 16 rules could retrieve themselves by name, and all 5 were the ones stored as separate records.

2.2 Rules as rows in the existing knowledge graph — rejected

Every other row in that table is 1:1 with its vector; a rule is 1:N, because one rule needs several independently embedded trigger situations. Mixed together this produces **top-K collapse**: measured on our data, one rule's five triggers occupied 4 of the top 5 result slots and only 2 of 3 rules appeared at all.

2.3 Score thresholding — rejected

We wanted a threshold that answers "does any rule apply here?". It does not exist in the vector space:

gate	positives	negatives	result
absolute similarity	0.409–0.681	0.342–0.494	**overlap**
margin (1st – 2nd)	0.0065–0.1820	0.0070–0.0407	**overlap**
grid search over both	—	—	best gate let **3 of 4** nonsense queries through

Similarity answers *"which rule is nearest"*. It cannot answer *"does any apply"* — that is not a property of the space. Zero has no representation in it.

2.4 Automatic contradiction detection — rejected

Asked whether two rules conflict in a given situation, the local model answered **one-sidedly**: with one phrasing it said "they conflict" 8 times out of 8; with another it said "they agree" 6 out of 6. Both scored 4/8 and 4/6 against ground truth, which is chance. We stopped after the second attempt rather than tune the prompt until the number looked right.

Consequence: contradiction between rules surfaces as a **cluster of override justifications a human reads**, not as an automatic check.

2.5 A background guard watching for violations — rejected

Static detection of the risky pattern before execution fired **2,797 times against 12 real mistakes** across three sessions — 254 false alarms per true positive. A warning at that rate is trained away within a day, which makes it worse than no warning.

2.6 Hooks for every rule — rejected

15 of 16 rules would have fired zero times, because the agents follow them unprompted. A hook that never fires is cost without signal.

2.7 An SMT/solver layer over the rule base — rejected

It has nothing to consume once automatic contradiction detection is out.

---

3. The finding that made retrieval work

A rule stored as its own wording cannot be found from the situation it governs.

storage form	rank-1 retrieval
the rule statement	**1/5** (and that hit was a lexical coincidence)
several short descriptions of triggering situations	**4/5**, then **10/10** on the full set

The reason is obvious in retrospect and easy to miss: a query issued during work is always a *situation* ("the probe returned zero"), never a *statement* ("suspect the measurement"). We were indexing the answer and querying with the question.

It is not a ranking problem, and that mattered for the design. Before the fix, the full top-5 for a real situation looked like this:

rank	score	record
1	0.5060	session notes, 2026-08-08
2	0.4850	session notes, 2026-08-09
3	0.4820	corpus-gap analysis
4	0.4830	*rule — contradiction is the signal*
5	0.4870	*rule — suspect the measurement* ← the correct answer

The spread between first and fifth is 2.4%. No amount of weight tuning wins inside a cluster that tight — the signal was not present at all. With a situational trigger the same rule scored 0.5808, roughly nine points clear of the whole field.

The one failure taught more than the four successes. One rule scored *worse* with its trigger than without it. Its triggers described "a library, a framework, a dependency"; the query described "validating inputs". That word was not in the trigger, so the trigger did not match.

A trigger is not magic. It is text, and it covers exactly the situations written into it. Three design consequences follow, and they are the core of the schema rather than footnotes:

- a trigger **cannot be one sentence** — it has to be a set of situations
- it must be **writable at runtime**, when a case appears that it did not cover
- and this is the mechanism by which a rule earns its keep: a rule that caught a real situation gets that situation appended; a rule that has caught nothing in a year has nothing to argue with

Hit counting is therefore a by-product of maintaining triggers, not a metric bolted onto a schema from outside.

Two secondary results shaped the schema:

- **Aggregation must be MAX, not MEAN.** Five short triggers scored 0.5547 under MAX and 0.4605 under MEAN — worse than a single long trigger (0.5053), because averaging dilutes each distinct situation into the others.
- **MAX does not reward trigger count.** A decoy rule with 12 unrelated triggers scored 0.5270 against the correct rule's 0.5547. More triggers buy coverage, not an unfair advantage.

Retrieval therefore does `DISTINCT ON (rule_id)` — collapse each rule to its single best-matching trigger *before* ranking — which is what prevents top-K collapse and requires the parent/child split.

---

4. Who decides "no rule applies"

Since no threshold separates *applies* from *does not apply*, we tested whether a model can state the null:

approach	correct
vector similarity alone (rank-1)	5/10
vector top-3, then the local model decides	**9/10**
— of which, nonsense queries correctly refused	**4/4**

The local model refused all four nonsense queries even though the vector layer had handed it a plausible-looking nearest rule each time. Cost is fixed at ~288 ms (retrieval) + ~1,900 ms (judgement); a cheap pre-filter does not work, because any cutoff that excludes noise also excludes real cases. That fixed cost is what makes this a **called** capability rather than a background process.

---

5. The question we had not asked: is the rule needed at all?

After ten rounds of measuring *findability*, the human made an observation that changed the study:

> *"*"The models we use already have their own values and constraints, and many are*

> *aligned with ours. We are tuning and complementing, not overriding. The value*
>
> *is in cases that are not straightforward, where the weights alone are not*
>
> *enough to decide."**

We had never tested that. Two tests followed.

5.1 Does the rule prevent a mistake? (cloud agent, retrospective)

Against real situations from one week, with transcripts showing what was actually done: **10 of 16 rules were followed unprompted**. Four were violated.

And the violations concentrate. Across three sessions, of 46 self-admitted mistakes, **35 (76%) belonged to a single rule** — and they share one mechanical shape, visible verbatim in the agent's own words: *a command piped through a filter, the filter swallows the error, and the empty output reads as success.*

On that first result the cloud agent — which is to say, this paper's co-author — concluded that *"a rule which tells me something I would do anyway is bloat."*

That conclusion was wrong, and the correction is the most instructive event in the study. The human objected on two grounds: it had been measured on a single subject and generalised to the system, and it had measured only *mistakes*, ignoring decisions that never become mistakes because the model chooses defensibly — just not the way the system intends.

Both objections were correct, and neither was a coding error. They were flaws in the experimental design, produced by the person who was not running the experiment.

5.2 Does the rule change the choice? (local agent, A/B)

The redesigned test measures decisions rather than errors. 16 situations, each run twice, with and without the rule in context:

already decides as intended	**10/16**
rule changed the decision	**6/16**
rule made the decision worse	**0/16**

That zero matters: a redundant rule costs memory, not behaviour.

Illustration. *"Your first fix failed. Will you try the same thing three more times with small variations?"*

- Without the rule: *"Yes — small changes can help identify the specific error."*
- With the rule: *"No."*

The "yes" is not a reasoning error. It is a defensible engineering opinion that happens not to be ours. This is precisely the case the human described: the rule does not correct stupidity, it settles a choice the weights leave open.

5.3 Rule value is per entity

The same test run on both agents diverged:

rule	local agent	cloud agent
suspect-the-measurement	knows it unprompted	**violated 35×**
map-before-deep-reading	**rule changed choice**	second nature (0 violations / 3 sessions)
state-results-without-framing	**rule changed choice**	second nature
frontmatter-or-it-is-not-indexed	**rule changed choice**	knows it **only because it is written down**

8 of 16 rules have value for at least one agent — double the figure obtained by measuring one subject.

This separates two axes that are easy to conflate:

- **KNOWING** — is it in the weights? (the local model is weaker)
- **DOING** — does it hold under load? (the cloud model is weaker: it knows suspect-the-measurement perfectly and broke it 35 times)

A rule helps on the first axis. On the second it does not; there, only an intervention at the moment of action helped.

---

6. The methodological result

After the A/B test we asked the model directly which of its rules carried real information and which were generic advice. It classified **7 of 8 as "just advice"** — including three that had measurably changed its decision minutes earlier:

rule	A/B outcome	model's own classification
read the live database, never a remembered number	**changed the choice**	"just advice"
map before deep-reading	**changed the choice**	"just advice"
state results without framing	**changed the choice**	"just advice"

A model can follow a rule without being able to judge one. This is the third independent instance of the same limit in this study, after contradiction detection and thresholding: the model reliably performs the object-level task and fails the meta-level question about it.

Had we trusted the classification, we would have deleted seven working rules.

Rule value is measured by behaviour, never by asking the model.

---

7. One failure that was the text, not the model

One rule genuinely failed the A/B test. It read:

> *"Tag hive records so a future session can find them: name, type, spirit,

> *importance, source."*

The local model concluded that frontmatter might be redundant if the information lived elsewhere — a sound inference from an incomplete premise. The rule never stated the operative fact: **a file without YAML frontmatter is silently skipped by the indexer and is never retrievable.**

Rewritten to state that, the decision flipped. The difference between advice and usable information is the difference between *"label your records"* and *"without this, it disappears."*

Criterion adopted: a rule must carry a fact that cannot be derived from general knowledge. If a competent model would reach the same conclusion without it, the rule is a record rather than an instrument — which is a legitimate purpose, but a different one.

---

8. Final design

```
rules (16)                    rule_triggers (55)            rule_overrides ├─ id, stat
```

Rules are **defeasible by design**: an `unless` field states when not following one is legitimate, and overrides are recorded as data with their justification rather than treated as violations. This follows established practice in defeasible/non-monotonic reasoning, where such an override is a *legitimate exception* logged with its justification path. Repeated overrides of one rule are how tension between two rules becomes visible.

Delivery differs per participant, deliberately:

participant	mechanism	why
local agent	one extra value on an existing retrieval verb	its tool surface is deliberately minimal; every added schema is prompt tax on a small model
cloud agent	an on-demand lookup	it already follows 10 of 16; reflexive consultation would be overhead
cloud agent	one automated check	for the single pattern it *knows* and violates anyway
application	HTTP endpoint	one source of truth for all three

Why the automated check sits after execution, not before. Before a command runs, the only visible fact is that a filter exists — true of 1,255 calls across three sessions. The signal exists only in the *result*: a filtered command that returned nothing narrows to 47, and excluding commands that report their own exit status leaves **11**, roughly four per session. Tested 6/6 on real cases; it warns and never blocks.

---

9. Results

metric	value
retrieval rank-1, 16 rules, production path	**10/10**
full gate (vector + model judgement)	**7/7**, including 3/3 correct refusals
rules changing the local agent's decision	6/16, **0/16 made it worse**
rules with value for at least one agent	8/16
latency	288 ms + ~1,900 ms
automated check: false positives in test	**0/4**, ~4 firings per session

9.1 The deployed system, read from the database

Everything above is measurement during the study. This is the state of the running system, queried while writing this section rather than recalled:

table	rows
`rules`	16
`rule_triggers`	55
`rules` carrying an `unless` clause	5
`rule_overrides`	1
trigger hits recorded	2

The two hits are the honest part of this paper. They are:

rule	the situation that fired it
`code-red-loud`	*"A health check reports green but the function does not work."*
`always-ground-truth`	*"You are claiming a result without fresh evidence."*

Two calls is not adoption. But the counter moved off zero without anyone instrumenting it to, and both firings were in exactly the class the design predicted: non-obvious situations where the weights alone give a defensible answer that is not the one this system wants.

9.2 The single override, in full

The override log has one entry, and it is worth reading because it is what the defeasible design is for:

> ***Rule:*** `always-ground-truth` *— no claim of done without fresh evidence.*
>
> ***Situation:*** *The backup was missing for a full day of work, and the human was*
>
> *two minutes from shutting the machine down.*
>
> ***What was done:*** *Ran the 10-minute backup instead of finishing quickly.*

> ***Reasoning:*** *The rule says verify with evidence; a second standing instruction*
>
> *says do not stall the human. Preference went to the backup.*

No solver detected that tension. It surfaced because a human wrote down why they went around a rule, in a table designed to accept that as data rather than treat it as a violation. If a second and third entry cluster on the same pair, that cluster *is* the missing sub-rule — which is the human-in-the-loop version of the automatic contradiction detection this study rejected at 4/8.

10. Limitations

- **Single system, two models.** Findings about *which* rules matter are specific to these agents; the methodological result (behaviour over self-report) is the part we expect to generalise.
- **Small n on the A/B test.** 16 rules × 2 conditions. The 0/16 "made it worse" is the most robust cell; the 6/16 split would move with more situations.
- **The retrospective test is self-reported.** It counts mistakes the cloud agent admitted in its own transcripts. Unadmitted mistakes are invisible to it by construction — the true violation count is a lower bound.
- **Usage is barely measured.** The `hits` column stands at 2 (§9.1). Two firings is a signal that the mechanism works end-to-end, not evidence that it is adopted. Whether these rules get consulted in live work is the open question, and it is deliberately left to data rather than assertion.
- **The counter itself was broken during part of the study.** An earlier round reported "0 hits" as a finding. The column was never incremented by any code path, so that zero was not a measurement — it was an unasked question wearing the appearance of an answer. Fixed and verified before the numbers above were taken. It is the same failure class as §7 and it happened *inside* the study that documents it.

11. What we would do next

Induce candidate rules from clusters of recorded lessons (inductive logic programming over the knowledge graph — 208 lessons, 189 in one cluster), gated by human approval, and let the `hits` telemetry decide which existing rules survive. Neither is built: we want to know whether the current 16 are used before adding more.

Appendix: how the study was run

The cloud agent conducted the measurements with itself as the first subject and the local model as the second. The human intervened twice in ways that changed the direction:

- *"Rules must earn their place"* → the `hits` mechanism and the override log.
- *"The models already have their own values"* → rounds 11–12, which turned the conclusion from "16 rules need delivering" into "8 have value, and which 8 depends on the agent."

The second intervention exposed a flaw in method rather than in code: the agent had generalised a conclusion from a single subject. Without that objection the system would look identical and its

justification would have been false.